

Quantització

Anna de Mier, Lluís Vena
anna.de.mier@upc.edu, lluis.vena@upc.edu

Departament de Matemàtiques • Universitat Politècnica de Catalunya

Abril 2026

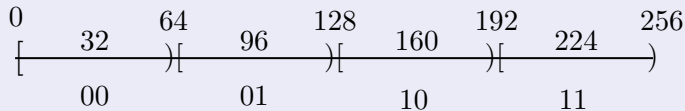
Capítols 9 i 10 de Introduction to Data Compression **Sayood, Khalid** Morgan
Kaufmann, 2012, 4th ed.

- 1 Quantització escalar
- 2 Quantització vectorial
- 3 Quantització de mètodes predictius

Quantització

Exemple

Imatge de grisos 8 bits $[0,256)$, codifiquem amb 2 bits:



Original

Uniforme: 2 bits por píxel



Quantització escalar

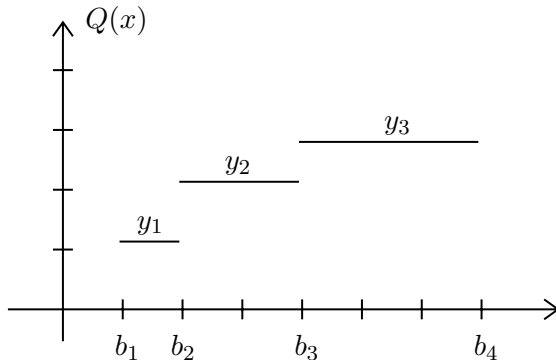


Figura: Funció de quantització $Q(x)$.

$\{b_i\}_{i=0,\dots,M}$ límits de decisió, $\{y_i\}_{i=1,\dots,M}$ nivells de reconstrucció,

$$Q(x) = y_i \text{ si } b_{i-1} \leq x < b_i$$

Quantització escalar: Problema general (I)

Problema: Modelitzem les nostres entrades amb una variable aleatòria X amb densitat de probabilitat $f(x)$.

Desitgem quantitzar una font donant els *límits de decisió* $\{b_i\}_{i=0,\dots,M}$ i els *nivells de reconstrucció* $\{y_i\}_{i=1,\dots,M}$ que determinen la funció de quantització $Q(x) = y_i$ si $b_{i-1} \leq x < b_i$.

Definim l'error (quadràtic)*:

$$\sigma^2 \equiv \int_{-\infty}^{\infty} [x - Q(x)]^2 f(x) dx = \sum_{i=1}^M \int_{b_{i-1}}^{b_i} [x - Q(x)]^2 f(x) dx \quad (1)$$

$\mathcal{C}$ codi amb M paraules de longituds $\{l_1, \dots, l_M\}$. Si $\tilde{l} = \sum_{i=1}^M l_i p(y_i)$, $p(y_i) = \int_{b_{i-1}}^{b_i} f(x) dx$, llavors

$$\tilde{l} = \sum_{i=1}^M \int_{b_{i-1}}^{b_i} l_i f(x) dx \quad (2)$$

*Hi ha altres *distàncies* o *errors*, σ , que usen $|x - Q(x)|$, $\sup |x - Q(x)| \dots$

Quantització escalar: Problema general (II)

Problema P_1 : Donat $\epsilon > 0$, una cota a l'*error*, trobar els límits de decisió $\{b_i\}$, els nivells de reconstrucció $\{y_i\}$ i les longituds de les paraules del codi $\{l_i\}$ que minimitzen (2) i satisfan $\sigma^2 < \epsilon$.

Problema P_2 : Donat $\delta > 0$, una cota a la longitud mitjana, trobar els límits de decisió $\{b_i\}$, els nivells de reconstrucció $\{y_i\}$ i les longituds de les paraules del codi $\{l_i\}$ que minimitzen (1) i satisfan $\tilde{l} < \delta$.

Quantització escalar uniforme

Tots els intervals són de la mateixa longitud, excepte potser els extrems:

$$x \in [0, L], \quad b_i - b_{i-1} = \Delta, \quad \Delta = \frac{L}{M}$$

Distribució uniforme $f(x) = \frac{1}{L}$

Minimitzar σ : $y_i = \frac{b_i + b_{i-1}}{2}$, $y_i - y_{i-1} = \Delta$ nivells de reconstrucció equiespaiats.

En ser una distribució uniforme podem utilitzar un codi de bloc de paraules de longitud n si $M = 2^n$:

$$\sigma^2 = \sum_{i=1}^M \int_{(i-1)\Delta}^{i\Delta} \left[x - \frac{2i-1}{2}\Delta \right]^2 \frac{1}{L} dx = \frac{\Delta^2}{12} = \frac{1}{12} \frac{L^2}{M^2}.$$

Duplicar el nombre d'intervals disminueix l'error a la meitat. En codis de blocs, augmentar en 1 bit la mida de les paraules redueix l'error a la meitat.

Quantització escalar uniforme adaptativa

Encara que $\Delta = b_i - b_{i-1}$ és constant, pot variar amb el *temps* depenent dels valors que va prenent la variable aleatòria.

FAQ Forward Adaptative Quantization: Es llegeixen blocs de N dades i es realitza la quantització basant-se en aquestes.

BAQ Backward Adaptative Quantization: S'actualitza Δ a mesura que es van llegint les dades.

Exemple

FAQ: D'una imatge es llegeixen blocs de 8×8 píxels, es calcula el seu màxim i el seu mínim i es quantitza en aquest rang $[\text{mín}, \text{máx}]$ (s'han de guardar aquests valors en codificar).

Exemple

BAQ. Jayant Quantizer: es varia Δ segons si les dades van caient en els nivells *interns* o *externs*; si és en els *interns* es disminueix Δ , si és en els *externs* s'augmenta Δ .

Quantització escalar no uniforme. Exemple μ -law

μ -law

És un mètode de quantificació *logarítmica* que s'usa per a senyals d'àudio: l'entrada es representa amb 14 bits i es realitzen una sèrie de transformacions per obtenir un codi de 14 bits en els quals s és el signe. Després se li aplica una quantització escalar no uniforme segons la taula:

14 bit Binary input code	Compressed code	14 bit Binary Output Code
s00000001abcdx	s000abcd	s00000001abcd1
s0000001abcdxx	s001abcd	s0000001abcd10
s000001abcdxxx	s010abcd	s000001abcd100
s00001abcdxxxx	s011abcd	s00001abcd1000
s0001abcdxxxxx	s100abcd	s0001abcd10000
s001abcdxxxxxx	s101abcd	s001abcd100000
s01abcdxxxxxxxx	s110abcd	s01abcd1000000
s1abcdxxxxxxxxx	s111abcd	s1abcd10000000

Quantització escalar no uniforme (I)

Tornem al cas general, considerem els problemes:

(A) Fixem $\{b_i\}$, minimitzar σ^2 respecte $\{y_i\}$. S'ha de complir:

$$y_i = \frac{\int_{b_{i-1}}^{b_i} x f(x) dx}{\int_{b_{i-1}}^{b_i} f(x) dx}$$

y_i centre de masses de l'interval.

(B) Fixem $\{y_i\}$, minimitzar σ^2 respecte $\{b_i\}$. S'ha de complir:

$$b_i = \frac{y_i + y_{i+1}}{2}$$

Els límits de decisió són el punt mitjà dels nivells de reconstrucció.

Quantització escalar no uniforme (II)

El problema general és trobar $\{b_i\}$ i $\{y_i\}$.

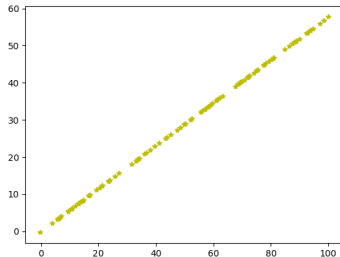
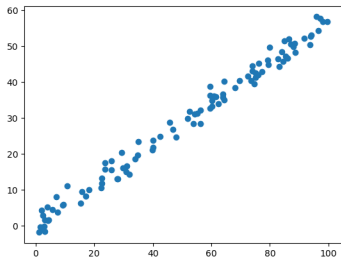
- 1 Fixem els valors inicials de $\{y_i\}$. Per **(B)** podem calcular la primera aproximació dels valors $\{b_i\}$. Ara usem **(A)** per actualitzar els valors $\{y_i\}$ i així successivament.
- 2 Suposem que volem trobar $b_0 = 0, b_1, b_2, \dots, y_1, y_2, \dots$
Donem un valor temptatiu a y_1^{**} , resoltem numèricament **(A)**
$$y_1 = \frac{\int_{b_0}^{b_1} x f(x) dx}{\int_{b_0}^{b_1} f(x) dx}$$
 on b_1 és la incògnita. Usant **(B)** obtenim y_2 , ara podem calcular b_2 resolent numèricament **(A)**...

^{**}El resultat dependrà del valor pres

Quantització vectorial (I)

Exemple

Quantitzar les components per separat pot suposar *malbaratar* bits:



Quantització vectorial (II)

Exemple

Codis correctors d'errors:

(0 0 0 0 0 0 0) 0000

(0 0 0 0 0 1 0) $\rightarrow$ (0 0 0 0 0 0 0) 0000

(1 0 0 0 0 0 0) $\rightarrow$ (0 0 0 0 0 0 0) 0000

(0 0 0 0 0 1 1) $\rightarrow$ (1 0 0 0 0 1 1) 1001

(0 0 0 0 1 0 1) $\rightarrow$ (0 1 0 0 1 0 1) 0101

(0 0 0 0 1 1 0) $\rightarrow$ (0 0 1 0 1 1 0) 0010

(0 0 0 1 0 1 1) $\rightarrow$ (0 0 0 1 1 1 1) 0001

(0 0 0 1 1 0 0) $\rightarrow$ (1 0 0 1 1 0 0) 1000

(1 1 0 0 1 0 1) 0101

(1 0 1 1 0 1 0) 1010

Quantització vectorial (III)

Diccionari compost per vectors $\{\vec{y}_i\}$ i codificar els nostres vectors amb el diccionari.

K-means – Linde-Buzo-Gray (LBG)

- 1 Inicialitzem un conjunt de vectors de reconstrucció $\{\vec{y}_i^{(1)}\}$, $k = 1$, $D^{(0)} = 0$, ϵ llindar.
- 2 Trobem les regions de quantització

$$V_i^{(k)} = \left\{ \vec{x}, d(\vec{x}, \vec{y}_i^{(k)}) < d(\vec{x}, \vec{y}_j^{(k)}) \forall j \neq i \right\}$$

- 3 Calculem la distorsió

$$D^{(k)} = \sum_{i=1}^M \int_{V_i^{(k)}} \|\vec{x} - \vec{y}_i^{(k)}\| f(\vec{x}) d\vec{x}$$

- 4 Si $\frac{D^{(k)} - D^{(k-1)}}{D^{(k)}} < \epsilon$ llavors **stop**.
- 5 $k = k + 1$, calculem els nous valors de $\{\vec{y}_i^{(k)}\}$, que són els centroides de $\{V_i^{(k)}\}$ i tornem al punt 2.

Quantització vectorial (IV)

Observacions:

- ❶ Quan s'usa per classificar objectes s'utilitza un conjunt per a entrenar i un altre de test.
- ❷ No sempre convergeix a la solució òptima.
- ❸ Depèn de l'elecció inicial de $\{\vec{y}_i^{(1)}\}$.
Una forma d'escollir-los és començar amb un vector que sigui el centroide. A continuació es pertorba per obtenir dos vectors, s'aplica l'algorisme, els nous centroides es tornen a “dividir” i així fins a obtenir el nombre de vectors desitjat.
- ❹ Depèn de les definicions de $d(\vec{x}, \vec{y})$ i $\|\vec{x}\|$ i de com s'actualitza.

Quantització de mètodes predictius (I)

Exemple

Original	0	1,2	2,4	4,9	7,1	6,3	4,8	3,4	0,9	-0,9	-2,4	-4,1
Diferències		1,2	1,2	2,5	2,2	-0,8	-1,5	-1,4	-2,5	-1,8	-1,5	-1,7
Quantitzat		1,5	1,5	2,5	2,5	-0,5	-1,5	-1,5	-2,5	-1,5	-1,5	-1,5
Recuperat		1,5	3	5,5	8	7,5	6	4,5	2	0,5	-1	-2,5
Error		0,3	0,6	0,6	0,9	1,2	1,2	1,1	1,1	1,4	1,4	1,6

Límits: ...-3,-2,-1,0,1,2,3...

Nivells -3,5, -2,5, -1,5, -0,5, 0,5, 1,5, 2,5, 3,5...

L'error va creixent a poc a poc.

Quantització de mètodes predictius (II)

$\{x_n\}$ la seqüència original

$\{d_n\}$ la seqüència de diferències, $d_n = x_n - x_{n-1}$

$\{\hat{d}_n\}$ la seqüència de diferències quantitzades, $\hat{d}_n = Q(d_n) = d_n + e_n$

$\{\hat{x}_n\}$ la seqüència reconstruïda

Calculem $\hat{x}_n = \hat{x}_{n-1} + \hat{d}_n$, prenent $\hat{x}_0 = x_0$:

$$\hat{x}_1 = \hat{x}_0 + \hat{d}_1 = x_0 + d_1 + e_1 = x_0 + (x_1 - x_0) + e_1 = x_1 + e_1$$

$$\begin{aligned}\hat{x}_2 &= \hat{x}_1 + \hat{d}_2 = (x_1 + e_1) + (d_2 + e_2) = \\ &= x_1 + (x_2 - x_1) + e_1 + e_2 = x_2 + e_1 + e_2\end{aligned}$$

...

$$\hat{x}_k = x_k + \sum_{i=1}^k e_i$$

Com que la mitjana de e_i serà 0, es pot pensar que l'error es cancel·larà a mesura que recuperem més elements de la seqüència, però a la pràctica no acostuma a ser així (passeig aleatori o del borratxo).

Quantització de mètodes predictius (III)

Per evitar aquest problema, la seqüència de diferències es construeix

$$d_n = x_n - \hat{x}_{n-1}$$

Exemple

Original	0	1,2	2,4	4,9	7,1	6,3	4,8	3,4	0,9	-0,9	-2,4	-4,1
Diferències		1,2	0,9	2,9	2,6	-0,7	-1,7	-1,6	-2,6	-1,9	-1,9	-2,1
Quantitzat		1,5	0,5	2,5	2,5	-0,5	-1,5	-1,5	-2,5	-1,5	-1,5	-2,5
Recuperat		1,5	2	4,5	7	6,5	5	3,5	1	-0,5	-2	-4,5
Error		0,3	-0,4	-0,4	-0,1	0,2	0,2	0,1	0,1	0,4	0,4	-0,4

Límits: ...-3,-2,-1,0,1,2,3...

Nivells -3,5, -2,5, -1,5, -0,5, 0,5, 1,5, 2,5, 3,5...

L'error és l'error produït en la quantització.

Quantització de mètodes predictius (IV)

$\{x_n\}$ la seqüència original

$\{d_n\}$ la seqüència de diferències, $d_n = x_n - \hat{\mathbf{x}}_{n-1}$

$\{\hat{d}_n\}$ la seqüència de diferències quantitzades, $\hat{d}_n = Q(d_n) = d_n + e_n$

$\{\hat{x}_n\}$ la seqüència reconstruïda

$$\hat{x}_0 = x_0$$

$$\hat{x}_1 = \hat{x}_0 + \hat{d}_1 = \hat{x}_0 + d_1 + e_1 = \hat{x}_0 + (x_1 - \hat{x}_0) + e_1 = x_1 + e_1$$

$$\hat{x}_2 = \hat{x}_1 + \hat{d}_2 = \hat{x}_1 + (d_2 + e_2) = \hat{x}_1 + (x_2 - \hat{x}_1) + e_2 = x_2 + e_2$$

...

$$\hat{x}_k = x_k + e_k$$

Quantització de mètodes predictius (V)

En aquest procés estem assumint que el següent valor és igual a l'anterior i codifiquem la diferència:

$$p_n = f(\hat{x}_{n-1}) = \hat{x}_{n-1}, \quad d_n = x_n - p_n, \quad \hat{d}_n = Q(d_n) \quad \hat{x}_n = p_n + \hat{d}_n$$

En general podem suposar:

$$p_n = f(\hat{x}_{n-1}, \hat{x}_{n-2}, \dots, \hat{x}_{n-N})$$

f depèn de $\hat{x}_i$ (no de x_i) perquè el descodificador només té accés a $\hat{x}_i$

Exemple

El valor d'un píxel Θ es pot predir a partir dels valors A, B, C,... ja

llegits, $\begin{array}{|c|} \hline \begin{array}{ccc} B & C & D \\ A & \Theta & \end{array} \\ \hline \end{array}$

- filtre lineal (mitjana ponderada dels píxels),
- filtre no lineal (mediana, moda, màxim, mínim...)